# Movement Quest Final Report

Mario Leonel Gutierrez Espinal
Team Fitness, solo team
CSC 4360 Project One
October 31, 2026

## Purpose

Movement Quest is a workout logbook. A user can save workout plans, record sets, and look back at completed workouts. The app keeps the data on the phone. It does not require an account or give health or exercise advice.

## Inputs and results

A plan has a title, equipment, an optional note and an effort value from 1 to 5. A set has an exercise, reps and an optional weight in pounds. The app shows the saved plans, sets, total reps and number of finished workouts. A user can change or delete a record when an entry is wrong.

## Screens

Plans shows the saved plans and filters them by equipment. Create or Edit Plan changes a plan. Workout starts a session and saves sets. History groups finished sets by session. Settings saves preferences and the dark-mode choice. Plans, History and Settings are the main tabs; the other two screens open from them.

## Code structure

Screens handle input and show the current values. Shared widgets such as WorkoutCard and SetRow display records. The WorkoutPlan and TrainingLog models convert records to and from maps. DatabaseHelper runs the database calls. The reps and weight validators are shared by Workout and History.

The main class relationship is:

```
WorkoutPlan 1 ---- 0..* TrainingLog
```

Each TrainingLog belongs to one WorkoutPlan through workoutId. A plan can have many logs. WorkoutPlan stores id, title, equipmentTag, planNote and effortPreference. TrainingLog stores id, workoutId, sessionKey, exerciseLabel, repetitions, weight, loggedAt and sessionFinished. DatabaseHelper reads and writes both types.

Saving a set follows these steps:

Check the form and stop if it is invalid or a save is already running. Create a TrainingLog from the input. Wait for DatabaseHelper to insert it into SQLite. If the screen is still mounted, add the saved set to the screen and clear busy. If saving fails, keep the input and show "Could not save. Try again."

Both text controllers are disposed when Workout closes. Checks after async operations prevent setState from running on a screen that has been disposed. There is not yet an automated test that leaves the screen during a save.

### Database

SQLite stores workout_plans and training_logs. Foreign keys are enabled when the database opens. Deleting a plan cascades to its logs after the user confirms. Updates and deletes use bound query parameters.

All sets in one workout share a sessionKey. Finish marks them as finished. COUNT(DISTINCT sessionKey) counts the finished workouts, so three sets in one session count as one workout. Reset removes only the unfinished sets in the current session.

Database version 2 adds an optional weight column with ALTER TABLE. Existing sets remain in place and their weight is null. SharedPreferences separately stores default effort, preferred equipment, dark mode and helper visibility.

### Gear Match

Gear Match uses fixed rules. It gives a plan 2 points for matching preferred equipment and 1 point for differing from the last finished plan. Missing preferences or history skip the related rule. The lower plan ID breaks a tie. The baseline uses only the equipment match.

The helper shows a reason and lets the user open a plan, dismiss the suggestion, or disable it. It never starts a workout. Its limit is that a saved equipment preference may differ from what the user wants today. AI_FEATURE.md records the five test cases, baseline comparison and this counterexample.

### Testing and results

On October 31, flutter analyze --fatal-infos reported no issues and flutter test passed all 20 tests. There are 5 Gear Match tests, 3 settings tests, 5 validator tests, 5 database/demo acceptance tests and 2 weight tests.

The database tests check session counting after deletion, cascade deletion, reset, close/reopen persistence, and demo isolation. One fixture uses sets of 12, 14 and 10 reps in the same session. The count stays 1 after deleting one set and becomes 0 after all three are deleted. The weight tests check save, edit, clear and upgrading a version 1 database without losing its saved set.

The earlier manual airplane-mode restart check passed on emulator-5554 using the demo build. Four plans, one finished session and 18 reps remained after restart. The normal release APK was later installed on emulator-5554 and opened to the Plans screen with the three regular starter plans. This release launch check does not replace the required second-device offline/settings test.

Validation rejects blank titles and invalid reps. Weight is optional; entered values must be positive and no more than 2000. Busy controls prevent repeated actions during a write. Save-error handling is in the code, but a deliberately failed database write has not been recorded as a tested result.

## Contribution and records

Team Fitness has one member, Mario Leonel Gutierrez Espinal. The barbell and optional-weight feature was his requested addition. The repository contains the app, test code and submission documents. Development assistance is recorded separately in AI_Usage_Log.md.

The weight feature and its migration tests are in commit 50875cd. Presentation files are in df76dc8, and the questions PDF is in 4bf8e0a. These are repository references, not proof of ClickUp task completion or historical board snapshots. The required ClickUp board records and contribution evidence PDF are pending.

## Limits and next step

The next app improvement would show the previous weight and reps for an exercise, such as "Last time: 135 lb for 5 reps." It would use existing local history and need a test with two sessions. Mario is the owner of this proposed task.